Что такое consumer group

1. Что такое consumer group и зачем она нужна

Consumer group в Kafka — это **группа consumers (консьюмеров)**, которые читают данные **из одного и того же топика совместно**, как единая команда.

Зачем нужна consumer group

Главная цель — **масштабировать чтение и обработку сообщений** и **не обрабатывать одно и то же сообщение несколько раз внутри одной логической обработки**.

Как это работает

- Топик в Kafka состоит из **партиций**
- Kafka распределяет партиции между consumers внутри **одной группы**
- **Одна партиция** → **только одному consumer в группе** (в один момент времени)

Пример

Есть:

- topic = `orders`
- 4 партиции
- consumer group = `order-processors`
- 2 consumer'а в группе

Тогда Kafka может распределить так:

- Consumer A: partition 0, 1
- Consumer B: partition 2, 3

Они читают **параллельно**, и обработка быстрее.

Что дает consumer group

1) Масштабирование (parallelism)

Можно добавить больше consumers в группу, чтобы быстрее обрабатывать поток.

2) Fault tolerance (отказоустойчивость)

Если один consumer упал:

- Kafka сделает **rebalance**
- его партиции перейдут другим consumer'ам в группе

3) Логическая “одна обработка”

Если все consumers в **одной группе**, то каждое сообщение (из конкретной партиции) обрабатывается **одним consumer'ом этой группы**, а не всеми сразу.

Важно понимать разницу

Если consumers в одной группе

- сообщения **делятся между ними**
- это как “workers”

Если consumers в разных группах

- **каждая группа** читает топик **независимо**
- одно и то же сообщение получат все группы

Пример:

- `billing-service` (group A)
- `analytics-service` (group B)

Обе группы читают `orders`, но каждая для своей задачи.

Ограничение по количеству consumers

Количество реально работающих consumers в группе **не может быть больше числа партиций** (для конкретного топика/набора топиков).

Например:

- 3 партиции
- 5 consumers в группе

Только 3 consumer'а будут читать данные, а 2 будут простаивать.

Коротко

Consumer group нужна, чтобы:

- распределять сообщения между несколькими consumers,
- обрабатывать их параллельно,
- переживать падение consumer'ов,

- и управлять независимым чтением через разные группы.

Если хочешь, дальше могу объяснить **offsets в consumer group** (это следующий ключевой шаг: как Kafka “помнит”, что уже прочитано).

2. Как Kafka назначает партиции консюмерам

Kafka назначает партиции консюмерам **внутри consumer group** через механизм **group coordination + partition assignment**.

Как это происходит по шагам

1) Consumer’ы входят в одну группу

У всех одинаковый `group.id`, например:

- `group.id=order-processors`

Они подключаются к Kafka и говорят:
“Я участник этой группы, хочу читать topic X”

---

2) Выбирается Group Coordinator

Для каждой consumer group Kafka использует специальный broker — **Group Coordinator**.

Он:

- отслеживает, кто в группе живой
- запускает **rebalance**
- фиксирует состав группы
- помогает согласовать назначение партиций

---

3) Происходит rebalance

Rebalance запускается, когда:

- consumer добавился
- consumer упал / отключился
- изменилось количество партиций
- consumer слишком долго не отправлял heartbeat (Kafka считает его “мертвым”)

Во время rebalance Kafka **перераспределяет партиции** между активными consumer’ами.

Кто именно решает, кому какая партиция достанется?

Не “рандомно”.

Kafka использует **partition assignment strategy** (стратегию назначения), например:

- **RangeAssignor**
- **RoundRobinAssignor**
- **StickyAssignor**
- **CooperativeStickyAssignor** (часто предпочтительнее в современных версиях)

То есть назначение делается по **алгоритму**, а не вручную.

Популярные стратегии (простыми словами)

1) Range

Партиции делятся “диапазонами” по топикам.

Пример (1 topic, 6 partitions, 2 consumers):

- C1 → p0, p1, p2
- C2 → p3, p4, p5

Минус: при нескольких топиках может быть перекос нагрузки.

---

2) Round Robin

Партиции раздаются по очереди:

- C1 → p0
- C2 → p1
- C1 → p2
- C2 → p3
  …

Обычно распределяет более равномерно.

---

3) Sticky

Старается:

- распределить **равномерно**
- и при этом **минимально двигать партиции** при rebalance

Это важно, потому что лишние перемещения = паузы + повторные прогревы/кэши.

---

4) Cooperative Sticky

Более “мягкий” rebalance:

- не отбирает сразу всё
- перераспределяет постепенно

Это снижает stop-the-world эффект при ребалансах.

Важное правило

Внутри **одной consumer group**:

- **одна партиция назначается только одному consumer’у**
- но один consumer может получить **несколько партиций**

Пример назначения

Есть:

- topic `orders` с 4 партициями (`p0..p3`)
- группа `order-processors`
- 3 consumer’a: `C1, C2, C3`

Kafka может назначить так:

- `C1` → `p0, p1`
- `C2` → `p2`
- `C3` → `p3`

Если `C2` упал → rebalance:

- `C1` → `p0, p1`
- `C3` → `p2, p3`

Что влияет на назначение партиций

1. **Количество партиций**
2. **Количество consumer’ов в группе**
3. **Стратегия assignor**
4. **Подписка consumer’ов** (на какие топики подписаны)
5. **Тип ребаланса** (eager vs cooperative)

Практический момент (очень важный)

Если у тебя:

- 10 consumers
- 4 партиции

то одновременно реально будут работать только **4 consumer’a** (остальные idle).

Коротко

Kafka назначает партиции через:

- **Group Coordinator**
- **rebalance**
- **assignment strategy (Range / RoundRobin / Sticky / CooperativeSticky)**

Цель:
распределить партиции между consumer’ами группы так, чтобы чтение было корректным и по возможности равномерным.

Если хочешь, следующим сообщением объясню **rebalance protocol (heartbeat, session timeout, poll timeout)** — это обычно самый важный кусок для собеседований и практики.

3. Почему один partition может обрабатываться только одним consumer'ом в группе

Потому что это **базовое правило Kafka для сохранения порядка и простого управления offset’ами внутри consumer group**.

Короткая суть

Если бы **одну и ту же partition** в **одной группе** читали **два consumer’a одновременно**, появились бы проблемы:

- ломается **порядок обработки**
- сложно/опасно управлять **offset**
- возникают **дубликаты / пропуски**
- усложняется модель отказоустойчивости

Главная причина №1: порядок сообщений (ordering)

Kafka гарантирует порядок **только внутри одной partition**.

Пример partition:

`M1 -> M2 -> M3 -> M4`

Если её читает **один consumer**, порядок сохраняется:

- обработал M1
- потом M2
- потом M3

Если читают **два consumer'а одновременно**:

- Consumer A взял M1
- Consumer B взял M2
- B обработал быстрее A

В итоге с точки зрения бизнес-логики может получиться:

- `M2` обработано раньше `M1` ❌

Для многих кейсов это критично:

- платежи
- статусы заказа
- обновления профиля
- события по одному пользователю/entity

Главная причина №2: offset management становится небезопасным

В consumer group offset хранится как позиция чтения для partition (условно: "до какого места группа дошла").

Если partition читает **один consumer**, всё просто:

- он обработал сообщения
- закоммитил offset

Если partition читают **два consumer'а**:

- кто коммитит offset?
- какой offset считать "правильным"?
- что если один обработал до 100, а второй только до 95?

Можно легко получить:

- **пропуск сообщений** (offset ушёл вперёд слишком рано)
- **повторную обработку** (offset откатился / не совпал)

Главная причина №3: простая и предсказуемая модель масштабирования

Kafka масштабирует чтение так:

- **параллелизм = количество partition**
- каждая partition = независимый последовательный поток

Это очень понятная модель:

- хочешь больше параллелизма → увеличивай partitions
- хочешь больше consumers → добавляй их, но не больше числа partitions (иначе часть будет idle)

Если разрешить нескольким consumers читать одну partition в одной группе, модель стала бы сложной и непредсказуемой.

Главная причина №4: rebalance и failover проще

Когда consumer падает:

- Kafka просто передаёт его partitions другим consumer'ам

Это легко, потому что у каждой partition **один владелец** в группе.

Если владельцев несколько одновременно:

- нужно координировать shared ownership
- решать конфликты коммитов
- синхронизировать состояние обработки

Kafka специально избегает этой сложности.

Важно: это ограничение только внутри одной группы

Одну и ту же partition могут читать **много consumers**, если они в **разных consumer groups**.

Пример:

- `billing-group` читает `orders`
- `analytics-group` читает `orders`
- `fraud-group` читает `orders`

Все читают ту же partition, но **независимо**, со своими offset.

Аналогия

Представь partition как **одну очередь задач с нумерацией**.

Внутри одной команды (consumer group) Kafka назначает **одного ответственного** за эту очередь, чтобы:

- задачи шли по порядку
- было понятно, где команда сейчас находится
- при смене сотрудника можно было продолжить с нужного места

Коротко

Один partition → один consumer в группе, потому что это обеспечивает:

- **порядок сообщений**
- **корректные offset'ы**
- **простую отказоустойчивость**
- **предсказуемый parallelism**

Если хочешь, следующий вопрос логично разобрать так: **"как тогда повысить throughput, если partition один, а обработка тяжёлая?"** (там есть важные паттерны для собеседования и практики).

4. Что такое rebalancing и в каких случаях он происходит

Rebalancing (ребаланс) в Kafka — это процесс **перераспределения партиций между consumer'ами внутри одной consumer group**.

То есть Kafka как бы говорит:
"Состав группы изменился (или что-то пошло не так) — надо заново назначить партиции."

Зачем нужен rebalance

Он нужен, чтобы:

- подключать новых consumers к обработке
- забирать партиции у "умерших" consumers
- перераспределять нагрузку
- учесть изменения в топиках (например, добавили partitions)

Что происходит во время rebalance (по сути)

1. Kafka фиксирует, что состав/состояние группы изменилось
2. Group Coordinator инициирует rebalance
3. Consumer'ы временно перестают нормально читать (полностью или частично — зависит от стратегии)
4. Выполняется новое назначение партиций
5. Consumer'ы продолжают чтение уже с новыми партициями

Когда происходит rebalance (самые частые случаи)

1) Новый consumer присоединился к группе

Пример:

- было 2 consumer'а, стало 3
  Kafka пересчитает назначение партиций, чтобы распределить нагрузку.

---

2) Consumer вышел из группы (или упал)

Пример:

- процесс завершился
- контейнер перезапустился
- приложение крашнулось

Kafka забирает его партиции и раздаёт оставшимся consumer'ам.

---

3) Consumer перестал слать heartbeat

Даже если процесс "вроде жив", Kafka может посчитать его мёртвым, если:

- consumer завис
- long GC pause
- сеть отвалилась
- поток обработки заблокировался так, что poll/heartbeat не успевают

Тогда coordinator исключает его из группы → rebalance.

---

4) Превышен `max.poll.interval.ms`

Если consumer слишком долго не вызывает `poll()`, Kafka считает, что он не справляется с обработкой.

Это типичный кейс:

- ты читаешь пачку сообщений
- очень долго обрабатываешь
- `poll()` не вызывается вовремя

Результат: consumer исключается из группы → rebalance.

---

5) Изменилось количество партиций в topic

Например:

- было 4 partition
- стало 8

Группа должна заново распределить новый набор партиций.

---

6) Изменились подписки consumer'ов

Если consumer начал/перестал подписываться на какой-то topic (в рамках группы), назначение надо пересчитать.

Почему rebalance может быть "болезненным"

Во время rebalance возможны:

- пауза в обработке
- рост latency
- временное накопление lag
- повторная обработка сообщений (если offsets неаккуратно коммитятся)
- "thrashing", если ребалансы происходят слишком часто

Виды rebalance (важно для практики)

1) Eager rebalance (старый/жёсткий подход)

- У всех consumer'ов сначала **забираются партиции**
- Потом назначаются заново

Минус: stop-the-world эффект.

---

2) Cooperative rebalance (более мягкий)

- Партиции перераспределяются **постепенно**

- Меньше пауз и лишних перемещений

Обычно используется с `CooperativeStickyAssignor`.

Как уменьшить лишние rebalance’ы (практика)

- не делать слишком долгую обработку в `poll`-цикле
- настроить `max.poll.interval.ms` под реальное время обработки
- следить за `session.timeout.ms` и `heartbeat.interval.ms`
- использовать cooperative assignor
- аккуратно делать graceful shutdown consumer’a
- избегать частых рестартов контейнеров
- если обработка тяжёлая — выносить её в worker pool, а `poll()` держать “живым”

Пример (простой)

Есть:

- topic `orders` (4 partitions)
- group `order-processors`
- Consumers: C1, C2

Назначение:

- C1 → p0, p1
- C2 → p2, p3

Подключается C3 → rebalance:

- C1 → p0
- C2 → p1
- C3 → p2, p3
  (точное распределение зависит от assignor)

Коротко

Rebalancing — это **пере-назначение партиций в consumer group**, которое происходит при изменении состава группы, таймаутах, падениях, изменении partitions или подписок.

Если хочешь, дальше можно разобрать **разницу между** `session.timeout.ms`, `heartbeat.interval.ms` **и** `max.poll.interval.ms` — это очень частый вопрос на собеседованиях и ключ к пониманию “почему у меня постоянные ребалансы”.

5. Разницу между:
a. eager rebalancing
b. cooperative (incremental) rebalancing

Отличный вопрос — это **очень важная практическая тема** для Kafka.

Разница в двух словах

- **Eager rebalancing** = “все отпускают партиции сразу, потом назначаем заново”
- **Cooperative (incremental) rebalancing** = “перераспределяем только то, что нужно, поэтапно”

---

a) Eager rebalancing

Как работает

При ребалансе **все consumers в группе сначала теряют свои партиции** (revoke all), даже если часть назначений могла бы остаться прежней.

Дальше Kafka делает новое назначение и раздаёт партиции заново.

Что это значит на практике

- обработка **останавливается у всех** (stop-the-world)
- даже “здоровые” consumers временно простаивают
- больше лишних перемещений партиций

Плюсы eager

- проще модель
- исторически старый/классический механизм
- совместим со старыми assignor-стратегиями

Минусы eager

- заметные паузы в обработке
- скачки latency
- рост consumer lag во время ребаланса
- больше шансов на лишние re-initialization (кэши, state, connections)
- особенно болезненно при частых рестартах/масштабировании

---

b) Cooperative (incremental) rebalancing

Как работает

Consumers **не отдают все партиции сразу**.

Kafka (через cooperative assignor) делает ребаланс **поэтапно**:

1. У части consumers забираются **только конфликтующие / нужные для переназначения** партиции
2. Остальные партиции продолжают обрабатываться
3. На следующем шаге назначения “доезжают” новым владельцам

То есть перераспределение идёт **инкрементально**, а не “всё обнулили и заново”.

Что это значит на практике

- меньше stop-the-world эффекта
- часть обработки продолжается во время ребаланса
- меньше лишних перемещений
- более плавное масштабирование / rolling restart

Плюсы cooperative

- меньше паузы
- ниже latency spikes
- меньше lag during rebalance
- лучше для production с частыми деплоями/автоскейлингом
- более “мягкое” поведение группы

Минусы / особенности cooperative

- логика сложнее
- ребаланс может занять **несколько фаз** (не всегда за один шаг)
- нужны совместимые assignor’ы (например, `CooperativeStickyAssignor`)
- при миграции со старых assignor’ов нужно аккуратно настроить клиентов

Главное отличие на примере

Было:

- C1: p0, p1
- C2: p2, p3

Подключается C3.

Eager

- C1 и C2 **сразу отдают всё**
- никто не обрабатывает какое-то время
- потом Kafka раздаёт:
  - C1: p0
  - C2: p1
  - C3: p2, p3 (примерно)

Cooperative

- C1 и C2 продолжают обрабатывать большую часть своих партиций
- Kafka забирает только те, которые реально нужно передать C3
- перераспределение идёт по шагам с меньшей паузой

Таблица-сравнение (по сути)

Eager

- Отзыв партиций: **все сразу**
- Пауза обработки: **высокая**
- Перемещений партиций: **больше**
- Простота: **выше**
- Production UX: **хуже при частых ребалансах**

Cooperative

- Отзыв партиций: **частичный**
- Пауза обработки: **ниже**
- Перемещений партиций: **меньше**
- Простота: **ниже (сложнее)**
- Production UX: **лучше**

Что обычно используют

В современных прод-сценариях часто предпочитают:

- `CooperativeStickyAssignor`

Потому что он:

- старается сохранить текущие назначения (sticky)
- и делает ребаланс мягко (cooperative)

Практический вывод для собеседования

Если тебя спросят **“почему cooperative лучше?”**, хороший ответ:

> Он уменьшает stop-the-world паузы при ребалансе, снижает latency spikes и lag, потому что не забирает все партиции у всех consumers сразу, а перераспределяет только нужные партиции поэтапно.

Если хочешь, следующим сообщением могу дать **короткий “интервью-ответ на 30 секунд”** + **пример конфигурации Spring Kafka / Java consumer** для cooperative rebalancing.

6. Что такое partition assignment strategies (Range, RoundRobin, Sticky)

`Partition assignment strategies` — это **алгоритмы, по которым Kafka распределяет партиции между consumer’ами внутри одной consumer group во время rebalance**.

Именно стратегия решает:

- кому какие партиции достанутся
- насколько равномерно распределится нагрузка
- сколько партиций будет “перетасовано” при ребалансе

Зачем они нужны

Одна и та же группа может иметь:

- несколько топиков
- разное число партиций
- разное число consumers

Можно распределять партиции по-разному, и от этого зависит:

- **баланс нагрузки**
- **стабильность при ребалансах**

- **время простоя / latency spikes**
- **количество лишних перемещений**

---

1) RangeAssignor (Range)

Идея

Kafka группирует партиции **по каждому топику отдельно** и делит их между consumers **диапазонами** (ranges).

Пример (один топик)

Топик `orders`, 6 партиций: `p0..p5`
Consumers: `C1`, `C2`

Range может назначить так:

- `C1` → `p0, p1, p2`
- `C2` → `p3, p4, p5`

Для одного топика выглядит нормально.

Проблема Range (важная)

При **нескольких топиках** может появиться перекос.

Пример

Топики:

- `orders` (3 partitions)
- `payments` (3 partitions)

Consumers:

- `C1`, `C2`

Range делит **каждый топик отдельно**:

- `orders`: `C1 -> p0,p1`, `C2 -> p2`
- `payments`: `C1 -> p0,p1`, `C2 -> p2`

Итог:

- `C1` получает **4 партиции**
- `C2` получает **2 партиции**
  => нагрузка может быть неравномерной.

Плюсы Range

- простой и понятный
- часто нормален для одного топика
- исторически широко использовался

Минусы Range

- может быть плохой баланс при нескольких топиках
- не минимизирует перемещения при rebalance

---

2) RoundRobinAssignor (RoundRobin)

Идея

Kafka берёт все партиции (обычно по подпискам группы) и раздаёт их consumer'ам **по очереди**.

Пример

Есть 6 партиций и 2 consumer'а:

- `C1 <- p0`
- `C2 <- p1`
- `C1 <- p2`
- `C2 <- p3`
- `C1 <- p4`
- `C2 <- p5`

Итог:

- `C1` → `p0,p2,p4`
- `C2` → `p1,p3,p5`

Обычно это даёт **более равномерное распределение**.

Плюсы RoundRobin

- хороший баланс по количеству партиций
- особенно полезен при нескольких топиках

Минусы RoundRobin

- может сильно "перетасовывать" назначения при rebalance

- больше movement partition'ов, чем у sticky-подхода
- больше потенциальных пауз/прогревов состояния

---

3) StickyAssignor (Sticky)

Идея

Sticky пытается одновременно сделать **две вещи**:

1. **Равномерно** распределить партиции
2. **Сохранить текущие назначения максимально**, чтобы при rebalance двигать как можно меньше партиций

То есть он "липкий" (sticky):
если партицию не нужно переносить — он старается оставить её у текущего consumer'а.

Почему это важно

При rebalance перемещение партиций = это часто:

- пауза
- повторная инициализация обработчика
- cold cache
- повторное чтение после revoke/assign
- всплеск latency и lag

Sticky уменьшает эти эффекты.

Пример (идея)

Было:

- `C1` → `p0,p1`
- `C2` → `p2,p3`

Подключился `C3`.

RoundRobin может перераспределить "красиво", но сильно поменять всё

Sticky постарается:

- оставить максимум у `C1` и `C2`
- передать `C3` только часть нужных партиций

Итог — меньше движений.

Плюсы Sticky

- хороший баланс
- меньше лишних перемещений при rebalance
- лучше для production, где важна стабильность

Минусы Sticky

- логика сложнее
- сам по себе Sticky ≠ cooperative (это отдельная ось; есть ещё `CooperativeStickyAssignor`)

Важное уточнение: Sticky vs CooperativeSticky

Это часто путают.

- **StickyAssignor** — стратегия распределения (баланс + минимум движений)
- **CooperativeStickyAssignor** — sticky + **cooperative/incremental rebalance** (мягкий ребаланс в несколько фаз)

То есть:

- Sticky отвечает **как назначать**
- Cooperative — **как проводить rebalance (eager vs incremental)**

Как выбрать (практически)

Range

Подходит, если:

- один топик
- простая схема
- не критичен идеальный баланс

RoundRobin

Подходит, если:

- много топиков
- нужен более равномерный баланс
- приемлемы более частые перестановки при ребалансе

Sticky

Часто лучший выбор, если:

- production-нагрузка
- важна стабильность
- хочется меньше churn при ребалансах

CooperativeSticky (часто лучший современный вариант)

Если:

- частые деплои / rolling restarts
- autoscaling
- важны низкие паузы при rebalance

Коротко для собеседования

- **Range** — делит партиции диапазонами по каждому топику; простой, но может давать перекос.
- **RoundRobin** — раздаёт партиции по очереди; обычно баланс лучше.
- **Sticky** — балансирует и одновременно минимизирует перемещения партиций при rebalance.

Если хочешь, следующим сообщением могу показать **на одном и том же наборе (2 topic × 3 partitions × 2 consumers)** точное распределение для **Range / RoundRobin / Sticky** — это очень хорошо “садится” в голову.

7. Почему ребалансировки считаются "дорогими" операциями в продакшене

Потому что ребаланс — это не просто “перераспределить партиции”, а **временная деградация всей consumer group** (иногда почти stop-the-world), которая бьёт по latency, throughput и стабильности.

Почему это “дорого” в проде

1) Пауза в обработке сообщений

Во время ребаланса consumers:

- перестают нормально читать/обрабатывать (полностью или частично)
- ждут новое назначение партиций

Итог:

- **throughput падает**
- **lag растёт**
- увеличивается **end-to-end latency**

---

2) Stop-the-world эффект (особенно при eager rebalance)

При eager-модели:

- у всех отзываются партиции (`revoke`)
- потом назначаются заново (`assign`)

То есть даже те consumers, которые могли бы продолжать работу, **вынужденно простаивают**.

---

3) Повторная инициализация локального состояния

После переназначения партиций consumer часто должен заново:

- прогреть кэши
- восстановить состояние обработчиков
- открыть/переиспользовать внешние ресурсы
- пересоздать контекст обработки

Если у вас тяжёлый consumer (DB, HTTP, stateful logic), это заметно.

---

4) Риск дубликатов / повторной обработки

На границе `revoke/commit/assign` легко получить at-least-once эффекты:

- сообщение обработано, но offset не успели закоммитить
- после ребаланса новый consumer прочитает его снова

Итог:

- **дубликаты**
- дополнительная нагрузка на downstream
- необходимость идемпотентности

---

5) Рост нагрузки на брокеры и координацию группы

Ребаланс — это дополнительный протокольный обмен:

- join group
- sync group

- heartbeats / coordinator interaction

При частых ребалансах (thrashing):

- coordinator перегружается
- группа “дёргается”
- система становится нестабильной

---

6) Каскадный эффект в микросервисах

Если ваш consumer — часть пайплайна:

- Kafka consumer → DB → another topic → downstream services

То пауза в одном месте вызывает:

- накопление lag
- burst после восстановления
- spikes в БД / API
- нестабильность SLA по всей цепочке

---

7) Потеря равномерности и временный дисбаланс

После ребаланса:

- часть consumers может получить больше “горячих” партиций
- часть — меньше
- система некоторое время “устаканивается”

Если нагрузка неравномерная по партициям, это особенно болезненно.

---

8) Частые ребалансы могут съедать продовую производительность

Самый плохой сценарий — **rebalancing loop / rebalance storm**:

- consumer долго обрабатывает → не успевает `poll()`
- исключается из группы
- ребаланс
- снова присоединяется
- снова ребаланс

В итоге сервис “жив”, но реально почти не обрабатывает поток.

Почему это особенно заметно в production

В проде обычно есть:

- высокий трафик
- SLA по задержке
- stateful processing
- внешние зависимости (DB, Redis, HTTP)
- rolling deployments / autoscaling

Поэтому даже короткий ребаланс может стоить:

- минут lag recovery
- дублей
- всплесков ошибок/таймаутов
- деградации пользовательских сценариев

Коротко

Ребалансировки считаются дорогими, потому что они вызывают:

- **паузы в обработке**
- **рост lag и latency**
- **лишние перемещения партиций**
- **повторную обработку / дубликаты**
- **дополнительную координационную нагрузку**
- **риск нестабильности при частых повторениях**

Что обычно делают в проде, чтобы удешевить ребалансы

- `CooperativeStickyAssignor`
- корректные `max.poll.interval.ms`, `session.timeout.ms`, `heartbeat.interval.ms`
- короткий `poll`-цикл (тяжёлую обработку — в worker pool)
- graceful shutdown
- idempotent processing
- аккуратный rolling deploy (не ронять всю группу сразу)

Если хочешь, следующим сообщением могу дать **чеклист “почему у нас частые ребалансы”** (по логам и параметрам Kafka/Spring Kafka).